Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07 Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 5

Дисциплина(модуль)/предмет МДК 02.02. Инструментальные средства разработки и отладки программного обеспечения

Тема: Проектирование стратегии ветвления Git Flow для распределенной команды разработки

Студент (ка)/учащийся: Иванов Егор Михайлович / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: Чудинова А.А. / _____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «10» октября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 МОДЕЛЬ ВЕТВЛЕНИЯ GIT FLOW.....4
2 АИС «ЦИФРОВОЙ УНИВЕРСИТЕТСКИЙ АРХИВ» И ОРГАНИЗАЦИЯ КОМАНДЫ.....5
2.1 Особенности системы и требования к процессу.....5
2.2 Зоны ответственности.....6
3 ПОШАГОВЫЙ СЦЕНАРИЙ ЖИЗНЕННОГО ЦИКЛА.....7
3.1 Подготовка: ветки main и develop.....7
3.2 Этап feature: параллельная разработка модуля.....7
3.3 Слияние в develop через pull request.....7
3.4 Этап release: месяц интеграционного тестирования.....8
3.5 Выпуск стабильного релиза в main.....9
3.6 Экстренное исправление: hotfix.....9
4 ГРАФ КОММИТОВ.....10
5 АНАЛИЗ БЕЗОПАСНОСТИ (APPSEC).....13
ЗАКЛЮЧЕНИЕ.....15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....16

ВВЕДЕНИЕ

Цель работы — закрепить на практике понимание классической модели ветвления Git Flow (ветки main, develop, feature/…, release/… и hotfix/…), научиться распределять зоны ответственности в командной разработке и проектировать жизненный цикл релиза программного обеспечения для автоматизированной информационной системы (АИС) [1].

Вариант задания определяется по номеру в списке группы. Номер — 10, вариантов пять, поэтому при циклическом распределении выбран вариант 5: АИС «Цифровой университетский архив». Задание варианта — спроектировать процесс разработки нового модуля шифрования файлов по стандарту Git Flow: разработчики создают ветки feature/crypto-module, вливают их в develop, а после месячного этапа интеграционного тестирования выпускают стабильный релиз.

Задачи работы:

– описать роли веток Git Flow и правила их создания и слияния применительно к АИС;

– распределить зоны ответственности участников распределённой команды;

– составить пошаговый сценарий жизненного цикла feature → develop → release → main, а также экстренного hotfix, с командами Git на каждом этапе;

– построить граф коммитов, иллюстрирующий параллельные потоки разработки;

– определить, на каком этапе жизненного цикла должен подключаться статический анализ кода (SAST) и проверка на уязвимости.

Сценарий не только описан, но и проверен: скрипт `pr5-gitflow/gitflow_demo.sh` в репозитории `shop-integration` создаёт учебный репозиторий и выполняет все команды раздела 3. Граф коммитов в разделе 4 — фактический вывод Git после выполнения скрипта.

1 МОДЕЛЬ ВЕТВЛЕНИЯ GIT FLOW

Git Flow — модель ветвления, предложенная В. Дриссеном [2]. В отличие от упрощённого GitHub Flow, где есть одна основная ветка и короткие ветки задач, Git Flow разделяет ветки по ролям. Благодаря этому одновременно можно разрабатывать новую функциональность, стабилизировать готовящийся релиз и срочно исправлять уже работающую версию. Роли веток приведены в таблице 1.

Таблица 1 – Роли веток Git Flow

Ветка	Назначение	Создаётся от	Вливается в	Время жизни
main	Только стабильные релизные версии; каждый коммит помечен тегом версии	—	—	Постоянная
develop	Интеграционная ветка: в неё стекаются все готовые доработки	main (один раз)	release/…	Постоянная
feature/<имя>	Разработка одной изолированной функции	develop	develop	До слияния функции
release/ <версия>	Стабилизация, тестирование, исправление мелких ошибок перед выпуском; новые функции запрещены	develop	main (с тегом) и develop	Период тестирования
hotfix/ <версия>	Экстренное исправление критической ошибки в рабочей версии	main	main (с тегом) и develop или открытый release	Часы, дни

Слияния выполняются с ключом `--no-ff` (без «перемотки»). Тогда в истории остаётся отдельный коммит слияния, по которому видно, что группа коммитов составляла одну функцию или один релиз, и при необходимости функцию можно отменить целиком одной командой `git revert -m 1`.

2 АИС «ЦИФРОВОЙ УНИВЕРСИТЕТСКИЙ АРХИВ» И ОРГАНИЗАЦИЯ КОМАНДЫ

2.1 Особенности системы и требования к процессу

АИС «Цифровой университетский архив» обеспечивает долгосрочное хранение документов, диссертаций и научных отчётов с многоуровневым цифровым подписанием [1]. Для этой системы характерны:

– высокая цена ошибки при работе с данными: повреждённый при шифровании файл или потерянный ключ означают необратимую потерю документа, хранящегося годами;

– редкие крупные релизы: новые модули выходят после длительного интеграционного тестирования, частые выпуски не нужны;

– требования информационной безопасности: модуль шифрования обрабатывает ключи, поэтому код проходит анализ защищённости, а ключи не должны попадать в репозиторий;

– необходимость поддерживать работающую версию: пока новый модуль тестируется, ошибки в текущей версии (например, в проверке подписи) исправляются без ожидания релиза.

Эти особенности хорошо соответствуют Git Flow: месячная стабилизация идёт в отдельной ветке release, не блокируя разработку следующих функций в develop, а срочные исправления проходят через hotfix в обход незавершённого релиза.

Модуль шифрования разрабатывается двумя параллельными функциями: `feature/crypto-module` — шифрование и расшифрование файлов (в сценарии используется отечественный алгоритм ГОСТ Р 34.12-2015 «Кузнечик» [5], включая потоковую обработку больших файлов) и `feature/key-storage` — хранение и ротация ключей шифрования. Новая версия системы — 2.0.0, текущая рабочая — 1.0.0. Нумерация версий — по правилам семантического версионирования [4].

2.2 Зоны ответственности

Таблица 2 – Роли участников и права на ветки

Роль	Зона ответственности	Права на ветки
Тимлид — релиз-менеджер	Создаёт и закрывает ветки release и hotfix, ставит теги версий, принимает решение о выпуске	Единственный, кто сливает в main; утверждает PR в develop
Разработчик 1	feature/crypto-module: алгоритм шифрования, потоковая обработка, модульные тесты	Свои feature-ветки; PR в develop; исправления в release по задачам тестирования
Разработчик 2	feature/key-storage: хранилище и ротация ключей; дежурство по hotfix	Свои feature-ветки; hotfix-ветки по поручению тимлида
Инженер по тестированию (QA)	План и проведение интеграционного тестирования на ветке release	Только чтение; заводит задачи на исправления
Инженер по безопасности (AppSec)	Настройка SAST и SCA, ревью криптографии, пентест релиза	Обязательный рецензент PR в модуле crypto
DevOps-инженер	Конвейер CI/CD, защита веток, сборка и развёртывание	Настройки репозитория

В настройках удалённого репозитория (GitHub, GitLab) для веток main и develop включается защита: прямой push запрещён, слияние возможно только через запрос на слияние (pull request) после успешной сборки, тестов, проверки SAST и одобрения рецензентов. Ветки feature именуются по шаблону `feature/<краткое-имя>`, сообщения коммитов — по соглашению Conventional Commits (feat, fix, test, docs, chore).

3 ПОШАГОВЫЙ СЦЕНАРИЙ ЖИЗНЕННОГО ЦИКЛА

3.1 Подготовка: ветки main и develop

В main находится рабочая версия 1.0.0 с тегом v1.0.0. Интеграционная ветка develop создаётся от неё один раз и публикуется на сервере [3]:

```
git switch main
git tag -a v1.0.0 -m "Релиз 1.0.0"
git switch -c develop
git push -u origin main develop --tags
```

При использовании расширения git-flow этот шаг выполняется командой `git flow init -d`. В Visual Studio — окно «Изменения Git» → меню ветки → «Создать ветку», источник — main.

3.2 Этап feature: параллельная разработка модуля

Разработчики создают ветки от актуального develop и работают независимо, каждый в своей ветке (01–09.09 на рисунке 1):

```
# Разработчик 1
git switch develop && git pull
git switch -c feature/crypto-module
git add src/crypto/cipher.py
git commit -m "feat(crypto): шифрование файлов"
git commit -am "test(crypto): модульные тесты"
git push -u origin feature/crypto-module
```

```
# Разработчик 2
 git switch develop && git pull
 git switch -c feature/key-storage
 git add src/crypto/keystore.py
 git commit -m "feat(keys): хранилище ключей"
 git commit -am "feat(keys): ротация ключей"
 git push -u origin feature/key-storage
```

Пока функция в разработке, разработчик регулярно подтягивает в свою ветку изменения develop (`git pull origin develop`), чтобы конфликты решались небольшими порциями. Ключи шифрования в репозиторий не добавляются: в `.gitignore` внесены файлы ключей и `.env`, а в коде используются только тестовые ключи.

3.3 Слияние в develop через pull request

Готовая функция вливается в develop только через запрос на слияние. При открытии PR автоматически запускается конвейер CI: сборка, модульные тесты, SAST, поиск секретов (раздел 5). Затем код просматривают тимлид и инженер AppSec. После одобрения выполняется слияние (10–11.09):

```
git switch develop && git pull
git merge --no-ff feature/key-storage   -m "Merge PR #12: feature/key-storage ->
develop"
```

```
git merge --no-ff feature/crypto-module -m "Merge PR #14: feature/crypto-module ->
develop"
git push origin develop
git branch -d feature/key-storage feature/crypto-module
git push origin --delete feature/key-storage feature/crypto-module
```

Эквивалент в git-flow: `git flow feature finish crypto-module`. На практике слияние выполняется кнопкой «Merge pull request» в веб-интерфейсе после прохождения всех проверок.

3.4 Этап release: месяц интеграционного тестирования

Когда в develop собраны все функции версии 2.0.0, тимлид создаёт ветку релиза и «замораживает» функциональность (14.09). Новые функции в release не добавляются: они продолжают вливаться в develop и войдут в следующий релиз.

```
git switch develop && git pull
git switch -c release/2.0.0
echo "2.0.0" > VERSION && git commit -am "chore(release): версия 2.0.0, заморозка
функционала"
git push -u origin release/2.0.0
```

Ветка release разворачивается на тестовом стенде с копией архива. План месячного тестирования приведён в таблице 3. Ошибки, найденные при тестировании, исправляются коммитами прямо в release (через короткие ветки и PR в release).

Таблица 3 – План интеграционного тестирования релиза 2.0.0

Неделя	Работы	Ответственный
1 (14–20.09)	Функциональное тестирование: шифрование и расшифрование документов всех типов, совместимость с электронной подписью	QA
2 (21–27.09)	Тестирование больших файлов и нагрузки; исправление ошибки на файлах больше 2 ГБ	QA, Разработчик 1
3 (28.09–04.10)	Полный SAST и SCA, ревью криптографии, тест на проникновение (пентест); исправление прав доступа к файлу ключей	AppSec, Разработчик 1
4 (05–12.10)	Регрессионное тестирование, проверка восстановления из резервной копии, CHANGELOG	QA, тимлид

```
git commit -am "fix(crypto): ошибка на файлах больше 2 ГБ"          # неделя 2
git commit -am "fix(keys): права доступа к файлу ключей"           # неделя 3
git commit -am "docs: CHANGELOG 2.0.0, тестирование завершено"     # неделя 4
git push
```

3.5 Выпуск стабильного релиза в main

После успешного завершения тестирования и прохождения всех проверок безопасности тимлид выпускает релиз (14.10). Ветка release вливается в main, коммит слияния получает тег версии, а затем release вливается обратно в develop, чтобы исправления, сделанные во время тестирования, не потерялись:

```
git switch main && git pull
git merge --no-ff release/2.0.0 -m "Merge release/2.0.0 -> main"
git tag -a v2.0.0 -m "Релиз 2.0.0: модуль шифрования файлов"
git switch develop && git pull
git merge --no-ff release/2.0.0 -m "Merge release/2.0.0 -> develop"
git push origin main develop --tags
git branch -d release/2.0.0 && git push origin --delete release/2.0.0
```

Эквивалент в git-flow: `git flow release finish 2.0.0`. Появление тега v2.0.0 в main запускает конвейер развёртывания в рабочую среду.

3.6 Экстренное исправление: hotfix

24.09, во время тестирования релиза, на рабочей версии 1.0.0 обнаружена критическая ошибка: подпись с истёкшим сертификатом принималась как действительная. Ждать окончания месячного тестирования нельзя, поэтому ветка hotfix создаётся от main, а не от develop: от main, чтобы в рабочую версию не попали непроверенные функции модуля шифрования.

```
git switch main && git pull
git switch -c hotfix/1.0.1
git commit -am "fix(sign): подпись с истёкшим сертификатом принималась как действительная"
echo "1.0.1" > VERSION && git commit -am "chore: версия 1.0.1"
git switch main
git merge --no-ff hotfix/1.0.1 -m "Merge hotfix/1.0.1 -> main"
git tag -a v1.0.1 -m "Hotfix 1.0.1"
git switch release/2.0.0                      # открыт релиз -> исправление вливаем в него
git merge --no-ff hotfix/1.0.1                # конфликт в VERSION -> оставляем 2.0.0
git checkout --ours VERSION && git add VERSION && git commit --no-edit
git push origin main release/2.0.0 --tags
git branch -d hotfix/1.0.1
```

По правилам Git Flow hotfix вливается в main и в develop, но если в этот момент открыта ветка release, исправление вливается в неё: develop получит его позже вместе с релизом (шаг 3.5). Конфликт в файле VERSION ожидаем, потому что обе ветки меняют номер версии. Он разрешается в пользу версии релиза.

4 ГРАФ КОММИТОВ

На рисунке 1 показана схема ветвления по сценарию раздела 3 (построена скриптом `pr5-gitflow/gitflow_graph.py`). Каждая ветка — горизонтальная линия на своей дорожке, время идёт слева направо по оси дат. Создание ветки и слияние показаны только вертикальными и горизонтальными отрезками: сплошная линия — создание ветки, штриховая со стрелкой — слияние с ключом --no-ff. Белые круги — обычные коммиты, чёрные — коммиты слияния, ромбы — коммиты с тегами версий в main. Видны три параллельных потока: две функции в разработке, затем релиз в стабилизации и одновременно hotfix для рабочей версии.

Рисунок 1 – Граф коммитов Git Flow для релиза 2.0.0 АИС «Цифровой университетский архив» (лист повёрнут)

Тот же граф в текстовом формате — фактический вывод команды `git log --graph --all --decorate` после выполнения скрипта `gitflow_demo.sh` (листинг 1). Хеши коммитов детерминированы: при повторном запуске скрипта они совпадают.

```
*   8dc49e2 2026-10-14 Merge release/2.0.0 -> develop (HEAD -> develop)
|\
| | *   e6098bd 2026-10-14 Merge release/2.0.0 -> main (tag: v2.0.0, main)
| | |\
| | |/
| |/|
| * | 6fdaf55 2026-10-12 docs: CHANGELOG 2.0.0, тестирование завершено
| * | 7066228 2026-10-02 fix(keys): права доступа к файлу ключей (замечание пентеста)
| * |   5cb7837 2026-09-25 Merge hotfix/1.0.1 -> release/2.0.0
| |\ \
| * | | a244028 2026-09-22 fix(crypto): ошибка на файлах больше 2 ГБ (найдено на интеграционных тестах)
| * | | 7b13d3c 2026-09-14 chore(release): версия 2.0.0, заморозка функционала
|/ / /
* | |   2852b53 2026-09-11 Merge PR #14: feature/crypto-module -> develop
|\ \ \
| * | | 9dbb3ca 2026-09-08 test(crypto): модульные тесты шифрования и расшифрования
| * | | 5b0b216 2026-09-05 feat(crypto): потоковое шифрование больших файлов (диссертации, сканы)
| * | | 9fa2c2d 2026-09-02 feat(crypto): шифрование файлов ГОСТ Р 34.12-2015 «Кузнечик» в режиме MGM
* | | |   6af33f2 2026-09-10 Merge PR #12: feature/key-storage -> develop
|\ \ \ \
| |/ / /
|/| | |
| * | | 0fc1243 2026-09-09 feat(keys): ротация ключей и журнал доступа к ключам
| * | | ed7be80 2026-09-03 feat(keys): хранилище ключей шифрования, ключи вне репозитория
|/ / /
| | * 0d91633 2026-09-24 Merge hotfix/1.0.1 -> main (tag: v1.0.1)
| |/|
|/|/
| * 5ad1776 2026-09-24 chore: версия 1.0.1
| * e34b4c6 2026-09-24 fix(sign): подпись с истёкшим сертификатом принималась как действительная
|/
* f7b3649 2026-09-01 feat: хранилище документов, электронная подпись (v1.0.0) (tag: v1.0.0)
```

5 АНАЛИЗ БЕЗОПАСНОСТИ (APPSEC)

Вопрос задания: на каком этапе жизненного цикла Git Flow АИС должен подключаться статический анализ кода (SAST) и проверка на уязвимости, и почему?

Ответ. SAST подключается уже на этапе слияния feature-ветки в develop — как обязательная проверка каждого pull request (подход «shift left»). Ветка release служит вторым, контрольным рубежом: перед выпуском в main проводится полное сканирование, анализ зависимостей и пентест. В main попадает только код, прошедший оба рубежа.

Обоснование:

– чем раньше найдена уязвимость, тем дешевле её исправить: на PR исправляет сам автор, пока код свежий, а уязвимость, найденная на неделе 3 релиза, тянет за собой повторное тестирование;

– develop — общая ветка: уязвимый код, попавший в неё, расходится по всем новым веткам разработчиков. Проверка на входе в develop этого не допускает;

– для модуля шифрования особенно важны ошибки, которые SAST находит хорошо: «зашитые» в код ключи и пароли, слабые или запрещённые алгоритмы, предсказуемый генератор случайных чисел, отключённая проверка сертификатов;

– на этапе release анализируется весь продукт целиком, а не отдельные изменения: проверяется взаимодействие модулей, сторонние библиотеки (SCA) и поведение работающей системы (пентест, DAST), что на уровне одного PR невозможно;

– hotfix проходит тот же обязательный SAST в сокращённом виде: срочность не отменяет проверку, ведь исправление уходит сразу в рабочую среду.

Таблица 4 – Проверки безопасности на этапах Git Flow

Этап	Проверки	Блокирует слияние
Локально (pre-commit)	Поиск секретов (Gitleaks), линтер — быстрая обратная связь разработчику	Коммит
PR feature → develop	SAST изменённого кода (Semgrep, SonarQube, PVS-Studio [8]), поиск секретов, правила для криптографии, модульные тесты, ревью AppSec	Да — при уязвимостях высокого и критического уровня
Ветка release	Полный SAST всего кода, SCA (уязвимые зависимости), пентест и DAST на стенде, ревью криптографии	Да — выпуск в main невозможен
hotfix → main	SAST изменённого кода и поиск секретов (сокращённый набор), ревью тимлида	Да
main (после выпуска)	Периодическое сканирование: новые уязвимости в уже выпущенных зависимостях	Нет — заводится hotfix

Такой порядок соответствует ГОСТ Р 56939-2024 «Разработка безопасного программного обеспечения» [6], который относит статический анализ исходного кода к обязательным мерам процесса разработки, и ГОСТ Р 71207-2024 [7], устанавливающему требования к самому статическому анализу. Для российской АИС, хранящей документы университета, в набор инструментов целесообразно включить анализатор, сертифицированный ФСТЭК России (например, PVS-Studio).

ЗАКЛЮЧЕНИЕ

В ходе практической работы для варианта 5 — АИС «Цифровой университетский архив» — спроектирована стратегия ветвления Git Flow для разработки модуля шифрования файлов. Определены роли веток, правила их создания и слияния, распределены зоны ответственности шести участников команды и правила защиты веток main и develop.

Составлен пошаговый сценарий жизненного цикла с командами Git: параллельная разработка в feature/crypto-module и feature/key-storage, слияние в develop через pull request, месячное интеграционное тестирование в release/2.0.0 по недельному плану, выпуск стабильного релиза v2.0.0 в main с обратным слиянием в develop, а также экстренное исправление hotfix/1.0.1 во время открытого релиза. Сценарий проверен скриптом, по результатам которого построен граф коммитов. Обосновано подключение SAST на этапе pull request в develop с повторной полной проверкой безопасности на ветке release перед выпуском в main.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Практическая работа № 5 «Проектирование стратегии ветвления Git Flow для распределенной команды разработки» : задание по МДК 02.02 / СПбИЭУ. – Санкт-Петербург, 2026. – Текст : электронный // ЭИОС СПбИЭУ. – URL: https://spbiem.tech (дата обращения: 10.10.2026).

2. Driessen V. A successful Git branching model. – URL: https://nvie.com/posts/a-successful-git-branching-model/ (дата обращения: 10.10.2026). – Текст : электронный.

3. Chacon S., Straub B. Pro Git. – 2-е изд. – Apress, 2014. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 10.10.2026). – Текст : электронный.

4. Семантическое версионирование 2.0.0. – URL: https://semver.org/lang/ru/ (дата обращения: 10.10.2026). – Текст : электронный.

5. ГОСТ Р 34.12-2015. Информационная технология. Криптографическая защита информации. Блочные шифры : национальный стандарт Российской Федерации. – Москва : Стандартинформ, 2015. – Текст : непосредственный.

6. ГОСТ Р 56939-2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования : национальный стандарт Российской Федерации. – Москва : Российский институт стандартизации, 2024. – Текст : непосредственный.

7. ГОСТ Р 71207-2024. Защита информации. Разработка безопасного программного обеспечения. Статический анализ программного обеспечения. Общие требования : национальный стандарт Российской Федерации. – Москва : Российский институт стандартизации, 2024. – Текст : непосредственный.

8. OWASP Foundation. Source Code Analysis Tools. – URL: https://owasp.org/www-community/Source_Code_Analysis_Tools (дата обращения: 10.10.2026). – Текст : электронный.